The Zeta System Developers Guide (DRAFT)

Robert Büssow and Wolfgang Grieskamp

July 23, 2000

1 Introduction

The Zeta system is devided into the *session system* (see Sect. 2.2), the shell (see Sect. 2.3) and the adaptors (see 2.4). The session is responsible for the coordinating tasks, in particular for the computation. The shell takes care of interaction wiht the user. The actual work is done by the adaptors. The perform transformations and offer commands (forms) to execute specific tasks. Any adaptor can be plugged into the system. The adaptor tells the system what services it provides and by that augments the functionality of the hole systems. Adaptors use functionality of other adaptors in asking the session for a specific content, the sessin then looks for adaptors that are able to compute this content.

2 Components of the Zeta System

2.1 Units

Specification documents are represented by *units* in the Zeta system. In order to comfortably support specification that are composed from different formalisms, the *content* of a unit can be depicted in different *content types*. The repository does not know these content types and thus does not understand the content of a unit. Therefore, for the repository, a unit is an atomic entity. The adaptors (see 2.4) are responsible of the providing and transforming the content. The content of units may have references to other units. These references are resolved by the adaptors.

At the moment, among the existing content types are *Zirp* for abstract Z and MZS syntax, *Postscript*, *GIF*, etc. (see e.g. `ZirpContentTypes`, `TextContentTypes`, `GraphicsContentTypes`).

References to units in documents consist only of the unit name, that unambiguously define the unit, but says nothing about the content type. The actual content of a unit is resolved, if some tool requests the content in a specific type. By that, different tools can obtain different contents types (e.g. the LaTeX formatter get Postscript whereas a type-checker may ask for abstract syntax).

Contents of units are computed by the *adaptors* (see 2.4). The adaptors differentiated between *source adaptors* and *function adaptors*. Source adaptors create units and some contents from external documents, whereas function adaptors use existing contents of one or more units to create new, dependent contents.

The repository itself does not care about the content types it manages. Thus, adaptors can put any content types into the repository. It is subject to proper configuration of the reposity that requested contents are available, i.e. if an adaptor requests some content, there has to be an other adaptor that delivers is able to deliver this content.

Units are represented by `Unit` objects. A specific content is addressed via a `Target` that combines a unit and a content type. Content types are denoted by the `ContentType` class.

2.2 The Session System

The session system (online API: `Session.pizza`) is responsible for:

- for the registration of adaptors (`registerAdatpor`),
- for the registration of shells and message listeners (`registerShell`),
- for the registration of interaction forms (commands),
- for the proper computation of unit contents,
- for caching already computed contents and keeping it persistent between session, and
- for keeping track of outdated contents.

Adaptors register at the repository in telling, what content types they are able to compute. When the content of a unit in a specific type is requested, the session looks for an adaptor that is able to produce this content and ask the adaptor for the content. For the computation, the adaptor itself, can ask the session for other content, it needs. This procdure is continued recursively until a source adaptor is reached the imports the content by itself. For each content that is requested during this procdure, the session first looks in its internal cache, if the content has already been computed and the result is still up to date. For the latter mechanism, adaptors annotate the content, they define, with its dependencies.

2.3 The Shell

The shell (online API: `Shell.pizza`) is responsible for the interaction with the user of the Zeta system. It provides maintenance commands for, e.g., adaptor registration and gives an interface for the commands provided by the adaptors. Usually, the shell is also responsible of listening to messages from the adaptors and adequately displaying them.

In principal, a shell can come in two flavors, with a command line interface or a graphical user interface. Commands, defined by the adaptors, are defined in a way, that they can be used from a command line as well as in a GUI (see 2.6).

The shell also provides means of configuring the system via configuration files.

2.4 Adaptors

Adaptors (online API: `Adaptor.pizza`, `DefaultAdaptor.pizza`) do the actual work in the Zeta system. In contrary to the heretofore mentioned concepts, such as units, session, and shell adaptors are not independent from content types, but work with specific content types of the units.

Conceptual, one can differentiate between *source*, *function*, and *interaction* adaptors. Nevertheless, adaptors can have both roles and therefore, this differentiation is not reflected in the APIs.

Source Adaptors

As source adaptor has to implement the following features:

- Check, whether a unit can be created by this adaptor and add the unit (`lookupUnit`).
- Provide initial contents for the unit (`tryCompute`).

- Provide information whether the initial contents is still up to date (`isUptodate`).

In order to identify a source adaptor that is to create a unit, the unit name may contain the name of an adaptor. Hence, consists of an optional source adaptor specifier, the actual name, and some optional additional information for the adaptor:

$$[< AdaptorSepcifier >:] < Name > [\# < AdaptorData >]$$

A source adaptor has to somehow map the unit name to its external source. For this, it can use the adaptor data that is included in the extended unit name or its own configuration. Assume, for example, a source adaptor that exteranl sources are regular files. It could use the pure unit names as file names for file, it searches in the current directory. It could now use the adaptor data to allow the specification of an alternativ directory. Furthermore, it could be configured to always use an other directory or a list of other directories. After an adaptor has identified the source, it is supposed to complete the unit name, in order to make further requests to this units unambiguous. A source adaptor may fail in providing some unit content for various reasons. It throws one of the following execeptions then:

- `AmbiguousName` if their are several possibilities to map the (extended) unit name to an external source.
- `CannotCompute` if no source of the unit that allows the creation of the requested contents, could be found.
- `ComputationFailed` the source could be unambiguously identified, but the conversion in the requested format failed.

Note that the adaptor is supposed the signal the reason of the failure through the diagnostic channel (see 2.5).

Function Adaptors

Function adaptors solely compute contents from existing. Function adaptors ask the session for the contents their need for their computation. The session takes care of initiating needed other adaptors and initiate recomputation if some sources have changed. If the computation of the requested content fails, function adaptors throw an `ComputationFailed` exception. Additionally, function adaptors pass through `CannotCompute` exceptions.

Interaction Adaptors

Interaction adaptors provide commands for the user that perform some computation and deliver a result. In fact, the commands initiate the computation of contents through the tool chains and are the visible interface of the user. Note however, that all adaptors may provide commands for configuration.

2.5 Diagnostics and Exceptions

For adaptors, the session provides four message channels. Messages are send via the `send` method.

- **Results:** Through the result channels, commands deliver the results of their computation. (`ResultMessage`)
- **Progress:** Through this channel, the user is kept up to date upon the ongoin activity of the tools. (`ProgressMessage`)

- **Diagnostics:** Errors and warnings that appear during the computations are signalled through the diagnostic channel. Diagnostic messages also contain a `Locator` that identify the location of the failure in the source and a `Category`. (`Diag`).
- **Debug:** Debugging messages are passed over through the debug channel. The debug channel is configurable, to show only some debug messages (`DebugtMessage`).

The session itself forwards these messages to `Listeners` that register at the session via `registerListener`.

[more]

2.6 Interaction Forms

[more]

3 The Zeta Aggregation Library

[more]

A An Example Adaptor

In order to connect a tools the the Zeta system, it is necessaryto implement an adaptor interface. This adaptor can then easily be plugged into the system (e.g. via tt plugin mypackage.MyAdaptor). We use the implementation of the *graphics adaptor* to show how an adaptor is implemented. The adaptor is developed with the noweb system. An adaptor has to implement the interface `zeta.session.Adaptor`. For convenience, their is a default implementation (`DefaultAdaptor`). This abstract adaptor give default implementation of some adaptor functions, particularely those that are only needed by source adaptors. The default adaptor also provides methods to send messages to the session.

4 ⟨* 4⟩≡

```
package zeta.tools.graphics;

/** Graphics Converter Adaptor.
 *
 * Adaptor for graphics format conversion.  Computes various bitmap formats
 * (currently PPM and GIF) from encapsulated postript.
 *
 * @author <a href="http://swt.cs.tu-berlin.de/~buessow">Robert Buessow</a>
 * @version $Id: GraphicsAdaptor.nw,v 1.6 2000/07/17 05:09:11 wg Exp $
 */

⟨Imports 5a⟩
public class GraphicsAdaptor extends DefaultAdaptor {
    ⟨PrivateVariables 5b⟩
    ⟨MyName 5f⟩
    ⟨Initialization 6a⟩
    ⟨Storage 7b⟩
    ⟨Computations 7f⟩
    ⟨Forms 11a⟩
}
```

A.1 Imports

5a ⟨*Imports* 5a⟩≡ (4)

```
import java.lang.*;
import java.awt.*;
import java.awt.image.*;
import java.awt.event.*;
import java.io.*;
import java.util.Date;
import zeta.content.graphics.GraphicsContentTypes;
import zeta.util.Name;
import zeta.util.FatalError;
import zeta.format.*;
import zeta.form.*;
import zeta.session.*;
import zeta.tools.util.Executor;
import zeta.tools.util.SessionUtil;
```

A.2 Variables

5b ⟨*PrivateVariables* 5b⟩≡ (4)

⟨*ConverterExecutables* 5c⟩
⟨*Executor* 5d⟩
⟨*ToolDirectory* 5e⟩

5c ⟨*ConverterExecutables* 5c⟩≡ (5b)

```
private File ppm2gif_exec;
private File eps2ppm_exec;
```

The `Executor` is an auxiliar class to execute system commands. We use it to executed the converters.

5d ⟨*Executor* 5d⟩≡ (5b)

```
private Executor executor;
```

Private directory for files. The adaptor requests a directory to store, e.g., generated gif images with `session.getToolDir()` and stores the returned directory (`session.getToolDir`).

5e ⟨*ToolDirectory* 5e⟩≡ (5b)

```
private File tool_dir;
```

A.3 Adaptor Name

Each adaptor has a name that can be obtained via the `getName` method. The name should be unique throughout the session. The name is also used to identify a source adaptor in a MSL.

5f ⟨*MyName* 5f⟩≡ (4)

```
private static final Name thisName = new Name("GRAPHICS");

public Name getName() {
  return thisName;
}
```

A.4 Initialization

Adaptors are initialized via their `initialize` method, rather then their constructors. During initialization adaptors have to register the content types they want to use and the forms they provide. Additionally, they do whatever they need to initialize.

6a ⟨*Initialization* 6a⟩≡ (4)

```
private static final String startup_msg =
  "installed graphics adaptor v1.02 (c) 1998 buessow@cs.tu-berlin.de";

public void initialize(Session session, StorageHandle handle) {
  super.initialize(session, handle);

    ⟨RegisterContentTypes 6c⟩
    ⟨SendStartupMessage 6b⟩
    ⟨ToolDir 6d⟩
    ⟨SetConverterExecutables 6e⟩
    loadData();
    ⟨RegisterForms 7a⟩
}
```

An adaptor should send some (progress) message during initialization to inform the user about the actual configuration of the system.

6b ⟨*SendStartupMessage* 6b⟩≡ (6a)

```
sendProgress(startup_msg);
```

After startup the session does not know anything about the available content types, therefore each adaptor is supposed to register the content types it uses. These are the types it requests or provides. An adaptor can use the standard types from `zeta.content` or define its own, which are only known to this adaptor itself or, additionally, to its *friends*.

6c ⟨*RegisterContentTypes* 6c⟩≡ (6a)

```
session.registerContentType(GraphicsContentTypes.eps);
session.registerContentType(GraphicsContentTypes.gif);
session.registerContentType(GraphicsContentTypes.ppm);
```

Get tool dir for created files. The session provides for each adaptor a directory, for files, the adaptor needs to create.

6d ⟨*ToolDir* 6d⟩≡ (6a)

```
tool_dir = session.getToolDir(thisName);
```

Get converter executables. Scripts and binaries that are executed by adaptors are supposed to reside in the `$ET/lib` directory. `session.getToolDir` provides this directory.

6e ⟨*SetConverterExecutables* 6e⟩≡ (6a)

```
if(System.getProperty("os.name").startsWith("Windows")) {
  ppm2gif_exec = new File(session.getLibDir(), "ppm2gif.bat");
  eps2ppm_exec = new File(session.getLibDir(), "eps2ppm.bat");
} else {
  ppm2gif_exec = new File(session.getLibDir(), "ppm2gif");
  eps2ppm_exec = new File(session.getLibDir(), "eps2ppm");
}
executor = new Executor(session, this);
```

Register this adaptor's commands to the shell:

7a ⟨*RegisterForms* 7a⟩≡ (6a)

```
session.registerForm(set_form());
session.registerForm(show_form());
```

A.5 Storage

Adaptors can keep their state persistent. For this, it defines a serializable class holding the persistent data. This data can then be read during initialization and written upon termination of the session.

For conversion of Postscript to PPM the user may provide additional parameters (e.g. for magnification etc.) These options can be set via the command 'graphics gs-options ¡ops¿' (see also `set_form`). The options are kept persistent, i.e. they are preserved between sessions.

Momentary, only this one variable is kept persistent. However, in order to easily support more persistent data, a storage class is declared.

7b ⟨*Storage* 7b⟩≡ (4)

⟨*StorageClass* 7c⟩
⟨*LoadData* 7d⟩
⟨*FlushData* 7e⟩

7c ⟨*StorageClass* 7c⟩≡ (7b)

```
private static class PersistentData implements Serializable {
  case PersistentData(String gs_options);
}

/** ... and a variable of this type is declared */
private PersistentData data;
```

Load persistent data. `session.load` returns `null` if no data is available.

7d ⟨*LoadData* 7d⟩≡ (7b)

```
private void loadData() {
  data = (PersistentData) session.load(handle);
  /* Check, if data was stored in the previous session: */
  if(data == null) {
      /* Initialize 'data' to default: */
      data = new PersistentData(null);
  }
  gs_options_state.string = data.gs_options == null ? "" : data.gs_options;
}
```

Store data. This method is called by the session upon termination (see `session.flush`).

7e ⟨*FlushData* 7e⟩≡ (7b)

```
public void flush() {
  session.store(handle, data);
}
```

A.6 Computations

7f ⟨*Computations* 7f⟩≡ (4)

⟨*ProvideComputedTypes* 8a⟩
⟨*TryToComputeContent* 8b⟩

The content types, the adaptor computes can be obtained with the `computedTypes` method. That an adaptor says it can compute some content type does not mean that it is able to compute this type for any unit nor does it mean that it is the only adaptor can computes this type. It only means that the session might ask the adaptor, if the content is to be computed.

8a ⟨*ProvideComputedTypes* 8a⟩≡ (7f)

```
/**
 * Computed content types.  Returns an array of content types that
 * can be computed by this adaptor. */
public ContentType[] computedTypes() {
  return new ContentType[] {
      GraphicsContentTypes.gif,
      GraphicsContentTypes.ppm
  };
}
```

Try to compute content. This method does the actual computions of contents. `tryCompute` either defines the content to compute and terminates or throws an exception. It throws `CannotCompute` if the adaptor could not compute the content but an other adaptor should be asked. This is usually the case if some source content, the adaptors needs, is not available. In this case an other adaptor could be able to compute the content from other sources. If the adaptor cannot compute some content because an error occured during computation, it sends a diagnostic message (with `sendDiag` for example) and throws `ComputationFailed`. This is for example the case, if the typechecker tries to compute `checked-absy` but the unit is not context correct (see `zeta.sessionAdaptor.tryCompute`).

8b ⟨*TryToComputeContent* 8b⟩≡ (7f)

```
public void tryCompute(Target target)
throws Failures.ComputationFailed, Failures.CannotCompute {
  /* Take time before actual compuation starts. */

  /* Compute PPM: */
  if(target.type.equals(GraphicsContentTypes.ppm)) {
    ⟨ComputePPM 9⟩
  } else if(target.type.equals(GraphicsContentTypes.gif)) {
    ⟨ComputeGIF 10⟩
  } else {
      /* the content type is not computed by this adaptor
       * (should not happen). */
    throw new Failures.CannotCompute(target);
  }
}
```

9 ⟨*ComputePPM* 9⟩≡ (8b)

```
/* Request encapsulated Postscript. */
File eps_file = (File) target.unit.query(GraphicsContentTypes.eps);
String n = target.unit.getName().getRepr();
int i = n.indexOf(":");
if(i >= 0 && i < n.length()) n = n.substring(i+1);
File ppm_file = new File(tool_dir, n+".ppm");

/* Excute conversion filter.  The <code>Executor</code> class
 * is used to execute the conversion filter.
 * @see zeta.tools.util.Executor
 */
Date define_at = new Date();
sendProgress("converting "+target.unit.getName()+" to PPM");
boolean result;
if(data.gs_options == null) {
  /* without Ghostscript options. */
  result = executor.exec(default_locator,
                         eps2ppm_exec.getPath(),
                         eps_file.getPath(),
                         ppm_file.getPath(),
                         "4711ZETAEND");
} else {
  /* with Ghostscript options. */
  result = executor.exec(default_locator,
                         eps2ppm_exec.getPath(),
                         data.gs_options,
                         eps_file.getPath(),
                         ppm_file.getPath(),
                         "4711ZETAEND");
}

if(result) {
  /* conversion was successful, add content to unit */
  target.unit.define(GraphicsContentTypes.ppm, // type
                     ppm_file,                 // content
                     true,                     // persistent
                     define_at,                // start of comput.
                     new Target[] {            // dependencies
                       new Target(target.unit,
                                  GraphicsContentTypes.eps) },
                     this);                    // myself
} else {
   /* The conversion has failed. (We already send some diagnostics) */
   throw new Failures.ComputationFailed(target);
}
```

10 ⟨*ComputeGIF* 10⟩≡ (8b)

```
/* Get ppm content of the unit. Note that this will call
 * this.tryCompute recursively. */
File ppm_file = (File) target.unit.query(GraphicsContentTypes.ppm);
String n = target.unit.getName().getRepr();
int i = n.indexOf(":");
if(i >= 0 && i < n.length()) n = n.substring(i+1);
File gif_file = new File(tool_dir, n+".gif");

/* Notify user that conversion starts (might take notable time) */
Date define_at = new Date();
sendProgress("converting "+target.unit.getName().getRepr()+" to GIF");

/* Use <code>Executor</code> to execute the converter.  The
 * executor will send diagnostics, if execution fails (a non-zero
 * exit code is returned). */
boolean result = executor.exec(default_locator,
                               ppm2gif_exec.getPath(),
                               ppm_file.getPath(),
                               gif_file.getPath(),
                               "4711ZETAEND");

/* Define content, if conversion was successful.  Defines for
 * <code>target.unit</code> the content type
 * <code>GraphicsContentTypes.gif</code> with actual content
 * <code>gif_file</code>,  that is persistent (<code>true</code>),
 * defined at <code>define_at</code>, and dependent from the
 * content type <code>GraphicsContentTypes.eps</code> of this unit.
 */
if(result) {
    target.unit.define(GraphicsContentTypes.gif, // type
                       gif_file,                 // content
                       true,                     // is persistent
                       define_at,                // now
                       new Target[] {            // dependencies
                         new Target(target.unit,
                         GraphicsContentTypes.ppm) },
                       this);
} else {
    /* The conversion has failed. (We already send some diagnostics) */
    throw new Failures.ComputationFailed(target);
}
```

A.7 Forms

The graphics adaptor offers two forms. `set_form` allows to configure the converters in providing additional arguments for ghostscript. `show_form` shows the GIF content of a unit in a AWT window. `show_form` uses the `ImgFrame` to show the image.

It is strongly recommended to initialize the forms via methods and initializer, since the latter are evaluated before `initialize` is executed.

11a ⟨*Forms* 11a⟩≡ (4)

⟨*SetForm* 11b⟩
⟨*ImageFrame* 12⟩
⟨*ShowForm* 13⟩

A form is defined with `TopLevelForm.make`. The first argument tells the Zeta commander where this form should appear. Here, the *Options* menu is choosen. The second argument is the actual form. Here, this is a sequence of two forms—a descriptive text `LiteralForm` and a text input `StringForm`. The `StringForm` uses the `StringState.State` `gs_options_state` to store the current context of the input field (i.e. `gs_options_state.string`). The third argument is a descriptive text and the fourth, the action to be performed. This action is executed each time the form is executed (e.g. the apply button is pushed).

This form will appear in the command line shell as a command called `gs-options` and in the Zeta commander in the `Options` menu as `Ghostscript Options`.

11b ⟨*SetForm* 11b⟩≡ (11a)

```
private StringForm.State gs_options_state = new StringForm.State("");

/** Form to set Ghostscript options. */
private TopLevelForm set_form() {
  return TopLevelForm.make(
    new TopLevelForm.Style.Config().setModal(false),
    new SeqForm(
      new LiteralForm("gs-options").describe("Ghostscript Options"),
      new StringForm(gs_options_state)
    ),
    "Set Ghostscript options for image conversion",
    fun(Session session, Locator locator)
    throws Failures.FormFailed -> void {
      data.gs_options = gs_options_state.string;
    }
  );
}
```

AWT Frame to display units as images (gif).

12 ⟨*ImageFrame* 12⟩≡ (11a)

```
private static class ImgFrame extends Frame {
  private Image image;
  ImgFrame(String title, Image image) {
    super(title);
    this.image = image;

    /* Call getWidth in order to register me as image observer.
     * imageUpdate will be called, each time more data of the
     * image is loaded. (AWT loads images asynchronously) */
    image.getWidth(this);

    /* delete frame, if window close was selected. */
    addWindowListener(new WindowAdapter() {
      public void windowClosing(WindowEvent event) { dispose(); }
    });
  }

  /* set frame bounds after image loading is completed: */
  public boolean
  imageUpdate(Image image, int flags, int x, int y, int w, int h) {
    if((flags & ImageObserver.ALLBITS) != 0) {
      /* image is loaded: */
      setBounds(0, 0, image.getWidth(this)+10, image.getHeight(this)+30);
      repaint();
      return false;
    }
    return true;
  }

  /* paint image: */
  public void paint(Graphics g) {
    g.drawImage(image, 5, 25, this);
  }
}
```

The `show_form` uses the `UnitForm` form to specify the unit to be shown. The choosen unit will be stored in `UnitForm.defaultState.unit`. Note that this form may fail. It throws `FormFailed` in this case after sending a diagnostic message.

13 ⟨*ShowForm* 13⟩≡ (11a)

```
private TopLevelForm show_form() {
  return TopLevelForm.make(
    TopLevelForm.Style.Function(),
    new SeqForm(
      new LiteralForm("show").describe("Show Image"),
      new UnitForm()),
    "Show unit GIF-content in separate frame",
    fun(Session session, Locator locator)
    throws Failures.FormFailed -> void {
      if(UnitForm.defaultState.unit == null) {
        session.send(thisName,
                     new Diag(locator,
                              Diag.Category.Error,
                              "no unit given"));
        throw new Failures.FormFailed("ShowImage");
      }
      try {
        Unit unit = UnitForm.defaultState.unit;
        File gif_file = (File) unit.query(GraphicsContentTypes.gif);
        Image image =
          Toolkit.getDefaultToolkit().getImage(
            gif_file.getPath().toString());
        Frame frame = new ImgFrame(unit.getName().toString(), image);
        frame.show();
      } catch(Failures e) {
        session.send(thisName,
                     new Diag(locator,
                              Diag.Category.Error,
                              "GIF content not available"));
        throw new Failures.FormFailed("ShowImage");
      }
    }
  );
}
```